

Rapport

Vilka delar som testats på respektive nivå:

Vad gäller enhetstester skrev jag 10st som testade relevant funktionalitet utifrån taskUtils. Testen täcker bland annat valideringslogiken för ogiltig inmatning (validateTitle) för att säkerställa att tomma strängar och mellanslag hanteras korrekt och ger rätt felmeddelande.

Jag utförde även test av komponenten TodoForm för att säkerställa att nödvändiga element (label, inmatningsfält och submit-knapp) renderas korrekt i DOM:en, samt att användaren kan skriva in text i fältet. Jag skrev två integrationstester som täcker API-integration & nätverksanrop. Först testades funktionen getStockholmWeather tillsammans med MSW (Mock Service Worker). Här testade jag integrationen mellan kodbasen och ett simulerat externt API för att verifiera att svarsdata (temperatur och väderkod) har rätt datatyper, samt att systemet hanterar serverfel (status 500) och nätverksbortfall. Mitt resonemang kring valet av dessa tester var att jag ville täcka så mycket av appens kärnfunktionalitet som möjligt och även inkludera en del edge cases.

Vilka risker som återstår:

Eftersom väderanropet är simulerat via MSW fungerar koden mot den nuvarande strukturen, men det kvarstår en risk för att Open-Meteo ändrar sin datastruktur i framtiden. Därtill eftersom testerna körs isolerat på enhets- och integrationsnivå saknas heltäckande End-to-End-tester som testar hela kedjan från webbläsare till datalagring i en verklig miljö.

Användning av AI:

Jag använder AI som en lärare exempelvis för att förklara begrepp inom testning på ett annat sätt vilket sparar tid. När jag stötte på problem med min kod och om jag inte kunde lösa det på egen hand bad jag AI exempelvis leta igenom min kod för fel. Jag reflekterade kring vilka källor AI modellen använde för att förklara begrepp, och reflekterade även kring om AI:ns åsikter var relevanta för koden i detta projekt.